التزايد و switch وحلقات التكرار

الدرس 12 — 1IOT، SST-ICT-01 — الإسناد المركب، ++x وx++، بنية switch، الحلقات for و while و do while

19/10/2026

1 موضوعات اليوم

موضوعان: **تزايد** (**incremento**) متغير و**حلقات التكرار** (**cicli di ripetizione**). وبينهما، لإكمال **بنى القرار** التي رأيناها مع `if`، توجد أيضًا بنية **switch**. وفي نهاية الدرس بعض الأمثلة المجربة مباشرة على **onlinegdb**.

2 التزايد

تزويد متغير يعني **إضافة كمية ما إليه**. لنبدأ بمثال:

```
int x;
x = 6;
x = x + 3;
```

كم تساوي x في النهاية؟ **9**. السبب هو عمل **علامة المساواة**، الذي قيل مرات كثيرة: تأخذ المساواة **القيمة** التي على اليمين وتضعها في المتغير الذي على اليسار. على اليمين يوجد `x + 3`: وقيمة x هي 6، إذن $6 + 3 = 9$، ويوضع 9 داخل x. **يُكتب فوق** المتغير بقيمة جديدة.

تعريف

العملية `x = x + a`، حيث a أي عدد (أو متغير آخر)، تسمى **تزايدًا**: نأخذ المتغير و**نضيف إليه قيمة**.

3 الإسناد المركب: =+ و=- و=* و=/

المبرمجون **كسالى**، ولسبب وجيه: **كلما كتبنا أقل احتمالية الخطأ**. هذا لا يعني أن الكتابة القليلة لا تخطئ أبدًا، لكن السطر الطويل فيه فرص خطأ أكثر من القصير. لذلك في C **صيغة أكثر إيجازًا** للتزايد:

```
x = x + a;
// يمكن كتابتها بشكل مكافئ:
x += a;
```

الرمز =+ يعني: **خذ x**، و**أضف** a و**احفظ النتيجة** مجددًا في x. مثلًا، مع `x = 7;` ثم `x += 3;` تصبح x مساوية **10**. انتبه: نتحدث عن التزايد عندما نضيف قيمة **إلى المتغير نفسه**؛ أما إذا كان يجب جمع **عدة قيم مختلفة** فلم يعد تزايدًا، بل **جمعًا** عاديًا.

ملاحظة

a مجرد طريقة للإشارة إلى **أي** **عدد**: قد يكون 3 أو 4 أو 12، أو متغيرًا. مثلًا، بعد `int a; a = 12;` تعطي كتابة `x = x + 12;` أو `x += 12;` **النتيجة** **نفسها**.

الصيغة نفسها تصلح للعمليات الأخرى أيضًا:

النتيجة	مثال	تكافئ	الصيغة المختصرة
x تصبح 10	x تساوي 7	`x = x + 3;`	`x += 3;`
x تصبح 17	x تساوي 20	`x = x - 3;`	`x -= 3;`
x تصبح 10	x تساوي 5	`x = x * 2;`	`x *= 2;`
x تصبح 5	x تساوي 10	`x = x / 2;`	`x /= 2;`

=- هو **تناقص** (**decremento**)، لكنه من **عائلة** **التزايد** نفسها. القاعدة العامة: عندما نكتب `x = x` ثم **عملية** ثم **عددًا**، يمكن **نقل** **العامل** أمام علامة المساواة و**حذف** **x** **الثانية**.

ملاحظة

توجد الصيغة نفسها أيضًا للعوامل **على** **مستوى** **البت**، مثل `x &= ...` بدلًا من `x = x & ...`: وتفيد في **إخفاء** **البيانات** (**mascheramento**)، الذي سنراه عند دراسة **firmware** والعمل على البتات المنفردة.

4 التزايد بمقدار واحد: x++ و x--

للزيادة بـ**1** والنقص بـ**1** فقط توجد صيغة **أقصر** **بعد**:

صيغة الواحد	الصيغة المختصرة	الصيغة الطويلة
`x++;`	`x += 1;`	`x = x + 1;`
`x--;`	`x -= 1;`	`x = x - 1;`

علامتا **زائد** **متتاليتان** تزيدان المتغير 1، و**علامتا** **ناقص** تنقصانه 1. مثلًا `x = 7; x++;` يجعل x مساوية **8**. وسنستخدم هذا كثيرًا جدًا في **الحلقات**.

تنبيه

++ تصلح **للزيادة** **بـ1** **فقط**. لإضافة 2 لا توجد x+++: نكتب `x += 2` أو `x = x + 2`.

5 x++ أم ++x: إنها مسألة متى

يمكن كتابة ++ **بعد** المتغير (x++) أو **قبله** (++x). وحدها على سطر تفعل الشيء نفسه: **تزيد** **1**. يظهر الفرق عندما **تُستخدم** القيمة أيضًا، كما في الإسناد. قارنا في القسم هذين الكودين:

```
int x;
x = 7;
int y;
y = x++;
```

```
int x;
x = 7;
int y;
y = ++x;
```

تعريف

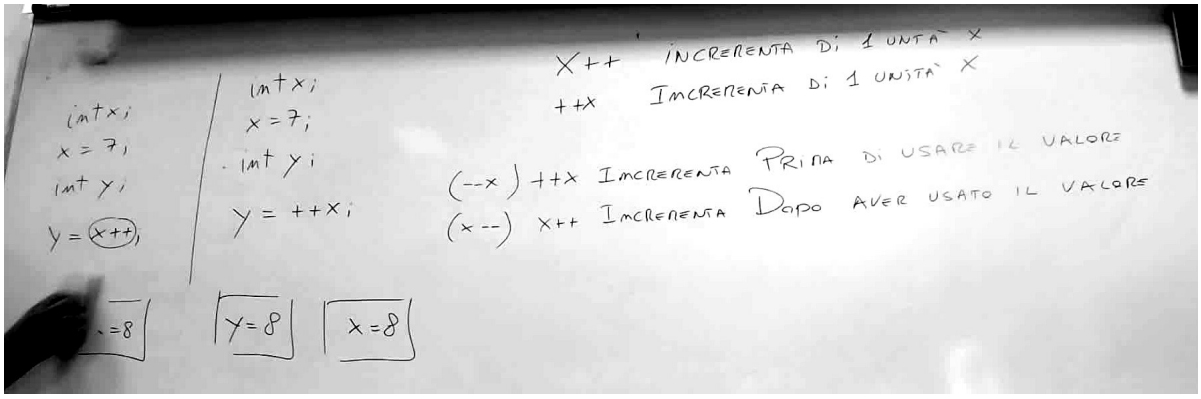
- ++x (و--x): **يزيد** **قبل** استخدام القيمة.
- x++ (و x--): **يزيد** **بعد** استخدام القيمة.

الكود	ماذا يحدث	النتيجة
y = x++;	ينسخ 7 إلى y أولًا، ثم يزيد x	y = 7, x = 8
y = ++x;	يزيد x إلى 8 أولًا، ثم ينسخه إلى y	y = 8, x = 8

في الحالتين **x** **في** **النهاية** **تساوي** **8**: يتغير فقط **ما** **ينتهي** في y.

تنبيه

في القسم **عكس** الشرح الأول الحالتين؛ و**صححه** الأستاذ **فورًا** على السبورة (الشكل 1). إذا كتبت في ملاحظاتك أن y مع y = x++; تساوي 8 أيضًا، **فصحح**: مع x++ تتلقى y القيمة **القديمة**، أي **7**.



شكل 1: السبورة مع الكودين والقاعدة الصحيحة: ++x يزيد قبل استخدام القيمة، و++x يزيد بعد استخدامها. (المربع تحت الكود الأول مغطى جزئيًا باليد.)

ملاحظة

معلومة طريفة: polling و interrupt. أثناء انتظار انتهاء القسم من النسخ، كان الأستاذ يسأل من حين لآخر "هل أتابع؟". في المعلوماتية تسمى هذه الطريقة **polling**: **نسأل** **جهازًا** **مرارًا** هل انتهى. والبديل هو **interrupt** (بالإيطالية **interruzione**، المقاطعة): الجهاز نفسه **ينبهنا** عندما ينتهي.

6 بنية switch

6.1 المشكلة: if كثيرة جدًا

تختار if الكود المنفذ بحسب **شرط**، لا يكون إلا **صحيحًا** **أو** **خطأ**. إذا طلب برنامج عددًا من 1 إلى 10 وكان عليه فعل 10 أشياء مختلفة بحسب العدد، فعلينا بـif كتابة سلسلة طويلة: `if (dato == 1) ...` ثم `else if (dato == 2) ...` ثم `else if (dato == 3) ...` وهكذا. هذا يعمل، لكنه **غير** **مريح** و**مربك**، ويسهل **الأخطاء**.

تعريف

switch بنية تختار **ماذا** **تفعل** بحسب **قيمة** **متغير**، بطريقة **منظمة**: فهي مثل if مع else كثيرة، لكن بتنسيق أوضح بكثير وأسهل في الإدارة.

6.2 البنية

```
switch (variabile_da_valutare)
{
    case opzione1:
        istruzione1;
        istruzione2;
        istruzione3;
        break;
    case opzione2:
        istruzione1;
        break;
    default:
        istruzione1;
        istruzione2;
}
```

- بين **قوسي** switch يوضع **المتغير** **المراد** **تقييمه** (**variabile da valutare**).
- كل **case** هو **قيمة** **ممكنة** للمتغير، تتبعها **نقطتان**. switch وcase هما **الكلمتان** **المفتاحيتان** للبنية.
- "istruzione" (تعليمة) **ليست** **أمرًا** في C: إنها تشير إلى **أي** **سلسلة** **من** **التعليمات**، واحدة أو اثنتين أو خمسين.
- كل case ينتهي بـ **break;**.
- **default** هي الإمكانية الأخيرة: تُنفذ عندما **لا** **يطابق** المتغير أي case.
- يمكن أن تكون حالات case **بقدر** **ما** **نريد**: 4 أو 7 أو 18 أو 25. على السبورة كانت اثنتين فقط لضيق المساحة.

6.3 المثال: قائمة بثلاثة خيارات

```c
int scelta;
printf("Inserisci una scelta");
scanf("%d", &scelta);

switch (scelta)
{
    case 1:
        printf("Hai scelto la prima opzione");
        break;
    case 2:
        printf("Hai scelto la seconda opzione");
        break;
    case 3:
        printf("Hai fatto la terza scelta");
        break;
    default:
        printf("Non hai fatto una scelta valida");
}
```

يقرأ البرنامج عددًا بـscanf. يقارن switch المتغير scelta (الاختيار) بالحالات، واحدة واحدة: مع 1، ومع 2، ومع 3. وعندما يجد مساواة ينفذ كود تلك الحالة. إذا أدخل المستخدم قيمة غير 1 أو 2 أو 3 تُنفذ default: "Non hai fatto una scelta valida" (لم تقم باختيار صالح).

ملاحظة

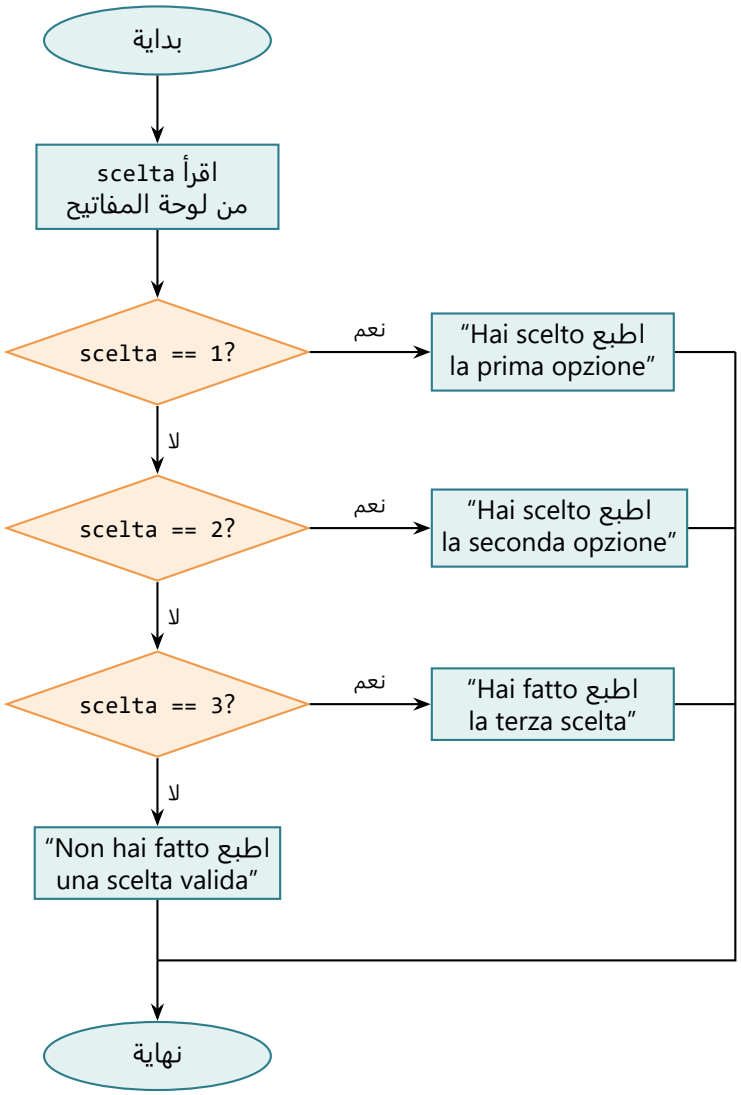
النص داخل printf يختاره المبرمج: الثالث مكتوب بشكل مختلف عن قصد، لنرى أنه لا يجب أن يكون متماثلًا دائمًا. في كل case تعليمة واحدة لضيق المساحة، لكن يمكن أن تكون أكثر من واحدة.

6.4 break: يقطع البنية

تعريف

break يقطع تنفيذ البنية: عندما تصادفه الآلة تخرج من switch وتتابع الكود الذي يأتي بعد قوس الإغلاق.

إذا كانت قيمة scelta هي 1، تنفذ الآلة printf("Hai scelto la prima opzione");، ثم تجد break فتخرج. لكن ماذا يحدث بدون break؟ بعد printf الأول تنزل الآلة: إيجاد case 2: لا يجعلها تعيد المقارنة، بل تتابع وتنفذ printf الحالة 2 أيضًا، وهكذا، حتى تجد break أو تصل إلى النهاية.



شكل 2: المخطط الانسيابي لـswitch في المثال: تُقارن الحالات واحدة واحدة؛ وbreak يقود فورًا إلى نهاية البنية.

تنبيه

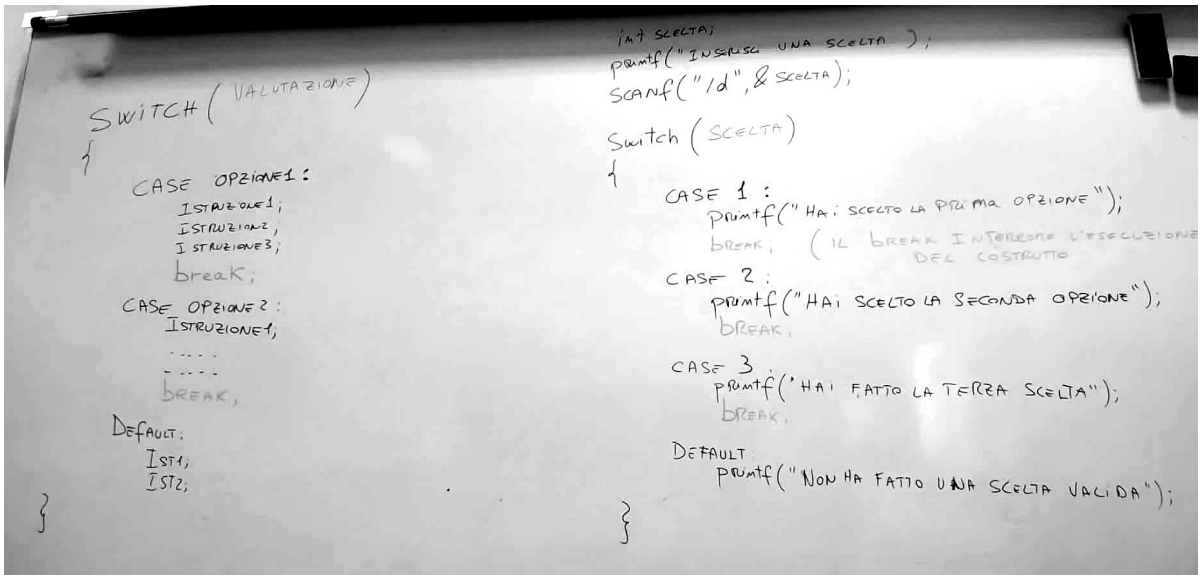
إذا **لم** **توضع** break، **ينفذ** switch **كل** **شيء** تحت الحالة التي وجدها. يجب الانتباه جيدًا: **توضع** **break** **دائمًا**، إلا عندما نريد هذا السلوك بالذات (انظر أدناه).

مثال

جمع عدة خيارات. يفيد هذا السلوك في فعل الشيء نفسه في عدة حالات. إذا أردت الكتابة نفسها عندما تكون scelta مساوية 2 **أو** ، أكتب حالتي case إحداهما تحت الأخرى:

```
case 1:
case 2:
    printf("Hai scelto o la prima o la seconda");
    break;
```

مع scelta تساوي 1 "تنزل" الآلة حتى printf؛ ومع scelta تساوي 2 تصل إليه مباشرة. وهكذا يصبح switch أداة قوية جدًا **للخيارات** **المتعددة**.



شكل 3: السبورة: على اليسار البنية العامة لـswitch، وعلى اليمين مثال القائمة بثلاثة خيارات والملاحظة "break يقطع تنفيذ البنية".

ملاحظة

الكل نعم، أو لا واحدة على الأقل. لمعرفة هل انتهى كل القسم من النسخ، أعطى الأستاذ مثالًا منطقيًا: "الجميع انتهوا" هي **AND**: الأول يقول نعم **و**الثاني يقول نعم **و**... والأخير يقول نعم. وعكسها، "لم ينته الجميع"، هي **OR**: الأول يقول لا **أو** الثاني يقول لا **أو**... بالانتقال من جملة إلى عكسها **تصبح** **AND** **هي** **OR** و**كل** **نعم** **تصبح** **لا**: وهذه **نظرية** **De Morgan**. وسنجدها في `do while`.

ألا يمكن كتابة "الجميع" ببساطة؟ يمكن الكتابة بطريقة أذكى، لكن الفحص لا بد منه: يجب إيجاد طريقة **لكتابة** **القليل**، لأنه كلما كتبنا أكثر زاد احتمال الخطأ. **الآلة** **لا** **تخطئ**: بل يخطئ **المبرمج**.

ملاحظة

لا تضعوا حدودًا لفائدة الأشياء. كان جد الأستاذ يتساءل لمن يفيد **جهاز** **التحكم** **عن** **بعد**: أما اليوم فلو طلب منا أحد أن نقوم لتغيير القناة لبدا ذلك جنونًا. حتى الأشياء التي تبدو بلا فائدة قد تصبح **مفيدة** **جدًا**.

7 حلقات التكرار

الحاسوب **لا** **يمل** من إعادة الأشياء نفسها، وهو بارع جدًا في إعادتها **متماثلة** **دائمًا**. تفيد **حلقات** **التكرار** في أن نقول له كرر عملية 10 أو 100 أو 1000 مرة. إذا كان علي إدخال 100 عدد، فبدلًا من كتابة `scanf` و`printf` مئة مرة أكتب بنية **تكرر** العملية نفسها 100 مرة.

تعريف

في C توجد **ثلاث** بنى للتكرار: **`for`** و**`while`** و**`do while`**. الأوليان، رغم اختلاف بنيتهما، **تفعلان** **الشيء** **نفسه**. والثالثة **شكل** **من** **`while`**.

for 7.1

```
for (condizione di partenza; condizione di ripetizione; modificatore)
{
    istruzioni da ripetere;
}
```

(**condizione di partenza** = شرط البداية، **condizione di ripetizione** = شرط التكرار، **modificatore** = المعدِّل، **istruzioni da ripetere** = التعليمات المكررة.) داخل القوسين **ثلاثة** **أجزاء**، تفصل بينها **الفاصلة** **المنقوطة**:

1. **شرط** **البداية**: من أين يبدأ العد؛
2. **شرط** **التكرار**: إلى متى نستمر؛
3. **المعدِّل**: كيف يتغير العد في كل دورة.

تنبيه

تفصل بين الأجزاء الثلاثة **الفاصلة** **المنقوطة**، **وليس** الفاصلة **وليس** النقطتان: وإلا يعطي المترجم **خطأ**. إنه خطأ سهل: فحتى على السبورة (الشكل 5)، في مثال الـ30 قفزة، جاءت فاصلة بدلًا من الفاصلة المنقوطة الثانية.

مثال

قفزات المتطوع. في القسم قام أحد الطلاب بـ10 قفزات، وهو يعد بصوت عالٍ. ماذا فعل غير القفز؟ **أبقى** **على** **العد**: القفزة الأولى، الثانية، الثالثة...وعندما وصل إلى عشرة **توقف**. وقرر بنفسه أن **يبدأ** **من** **واحد**: لم يقل له أحد ذلك. لو بدأ العد من 90، أو من 99 وعليه الوصول إلى 100، لاحتُرمت قاعدة "توقف عند 100"، لكن القفزات ستكون قليلة جدًا: لهذا يجب تحديد **نقطة** **البداية**. هذه هي أجزاء for الثلاثة:

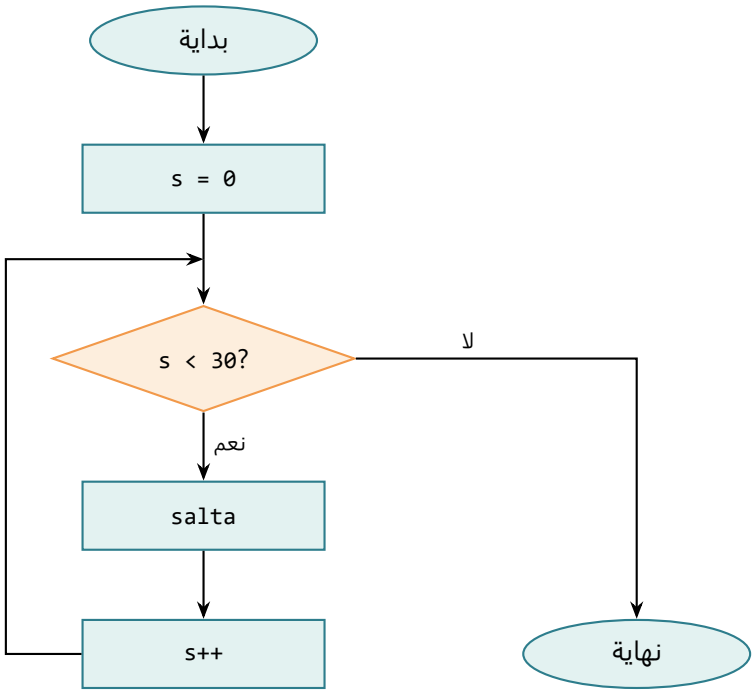
- **نقطة** **بداية** للعد، تُحفظ في **متغير**؛
- **إلى** **متى** نكرر: ما دام العد أصغر من نقطة الوصول نستمر في القفز؛
- بكم **نزيد** العد في كل قفزة: بواحد.

لنكتبها بلغة C، بـ30 قفزة والمتغير s (من "salti" أي القفزات؛ و**salta** = اقفز):

```
for (int s = 0; s < 30; s++)
{
    salta;
}
```

- `int s = 0`: **ينشئ** المتغير s ويعطيه **قيمة** **البداية**. نبدأ من **الصفر**: المعلوماتيون **يعدون** **من** **الصفر**، لأنه قبل البدء لم تتم أي قفزة. يمكن أيضًا إنشاء المتغير قبل for (`int s;`) وكتابة `s = 0` فقط.
- `s < 30`: **نكرر** **ما** **دام** s أصغر من 30.
- `s++`: في كل دورة **يزيد** s **بمقدار** **1**. يمكن أيضًا كتابة `s = s + 1` أو `s += 1`؛ وللقفز 2 في كل مرة `s += 2`.
- `salta;` تمثل **أي** **تعليمة** مكررة.

تتم القفزة و s قيمته 0، 1، 2، 3، ... حتى 29: إنها **30** **قفزة**. وعندما يصبح s مساويًا 30 يكون الشرط `s < 30` خاطئًا و**تنتهي** الحلقة.



شكل 4: المخطط الانسيابي لحلقة الـ30 قفزة. هو نفسه لـ`for` ولـ`while` في القسم 7.3: تتغير طريقة الكتابة فقط.

7.2 انتبه إلى البداية والخطوة

مثال

كم مرة تقفز هذه الحلقة؟

```
for (s = 7; s < 30; s += 3)
{
    salta;
}
```

كثيرون ينظرون إلى 30 فقط ويجيبون "30 مرة". **هذا غير صحيح**: العد **يبدأ من 7** ويتقدم **3 في كل مرة**.

s	7	10	13	16	19	22	25	28	31
s < 30?	نعم	نعم	نعم	نعم	نعم	نعم	نعم	نعم	**لا**

تقفز **8 مرات**. إنه مثال متطرف، فعمليًا نبدأ تقريبًا دائمًا من 0، لكنه يبين أن **الأجزاء الثلاثة** معًا هي التي تحدد عدد التكرارات.

تنبيه

الشرط هو شرط **مواصلة التكرار**، **وليس** شرط التوقف.

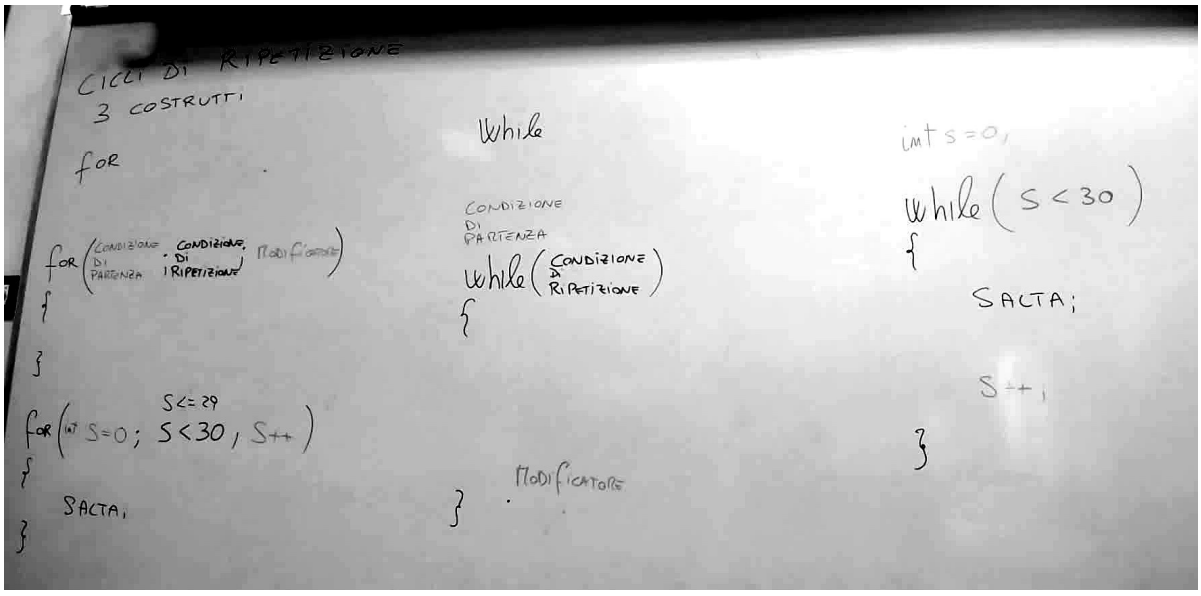
- `s < 30` و`s <= 29` **متكافئان** (الثاني فيه حرف زائد فقط، والمبرمج كسول).
- `s == 29` **خطأ**: لن يكرر إلا عندما يساوي s القيمة 29، إذن، بالبدء من 0، **أبدًا**.

7.3 while

`while` تفعل **الشيء نفسه** الذي تفعله for، مكتوبًا بطريقة مختلفة: شرط البداية **قبل** الحلقة، وشرط التكرار **بين القوسين**، والمعدِّل **داخل** الحلقة.

```
int s = 0;
while (s < 30)
{
    salta;
    s++;
}
```

"While" تعني **"بينما"**، "ما دام": **ما دام** s أصغر من 30، كرر. في كل دورة تعود الآلة إلى الأعلى و**تسأل** مجددًا هل `s < 30`؛ وعندما يصبح s مساويًا 30 **تخرج**. يوضع المعدِّل `++s` عادة **في النهاية**، لكن يمكن نقله بحسب السلوك المطلوب للحلقة. والمخطط الانسيابي هو الذي في الشكل 4.



شكل 5: السبورة: على اليسار for (الصيغة العامة ومثال الـ30 قفزة، مع `s <= 29` كبديل)، في الوسط الصيغة العامة لـ`while`، وعلى اليمين المثال. الألوان نفسها تشير إلى الأجزاء المتقابلة.

7.4 for أم while؟ مسألة أسلوب

يمكن استخدام **ما نفضله**: كل ما يُفعل بـ`for` يمكن فعله بـ`while`، والعكس. لكن هناك **ميلًا أسلوبيًا**:

تعريف

- `for`: عندما نعرف **مسبقًا** كم مرة نكرر، أي **يقرر** **المبرمج** (نفذ 10 قفزات: دائمًا 10).
- `while`: عندما **لا** **نعرف** كم مرة نكرر، لأن ذلك يعتمد على **ما** **يحدث** أثناء عمل البرنامج، وربما **خارج** النظام (اقفز حتى تتعب: لا نعرف كم قفزة ستكون).

مثال

- جعل مصباح **يومض** **10** **مرات** للإشارة إلى خطأ: من حيث الأسلوب `for`.
- جعل مصباح يومض **حتى** **يضغط** المستخدم **زرًا**: `while`، لأننا لا نعرف كم مرة سيومض.

مثال

الجمع بين الحلقتين. "نفذ مجموعات من 30 تمرين ضغط مع استراحة بين المجموعات، واستمر هكذا حتى تنهار": `while` خارجية ("استمر حتى تنهار") وفي داخلها `for` ("نفذ 30 تمرين ضغط")، مع الاستراحة. يمكن الجمع بين `for` و`for`، أو `while` و`while`، أو `for` و`while`: **كل** **ما** **نحتاج**، بعد فهم الأجزاء الثلاثة وطريقة ضبطها.

ملاحظة

برمج الأستاذ 25 سنة مستخدمًا `for` فقط؛ ثم تعلم استخدام `while` عند الحاجة. معرفة الاثنتين مهمة، لأن المبرمج يجب أن يقرأ أيضًا **كود** **الآخرين**: ففي فريق العمل هناك دائمًا من يفضل هذه ومن يفضل تلك.

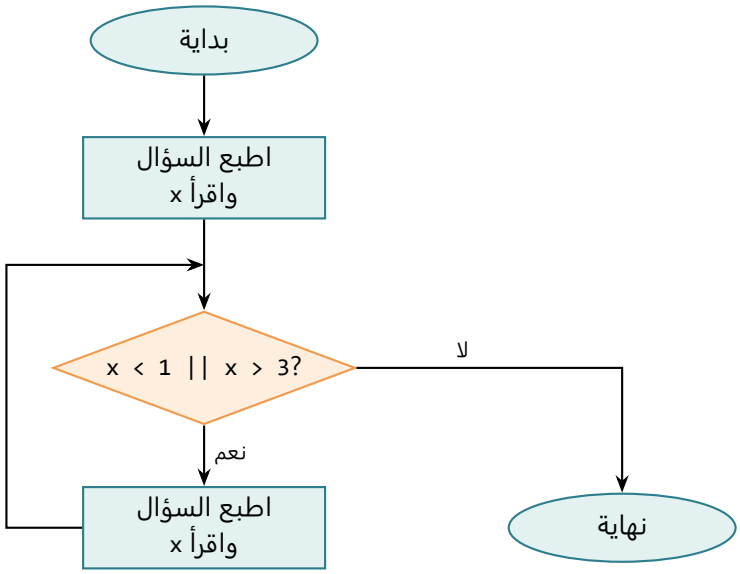
7.5 do while

المشكلة. لنكتب برنامجًا يطلب من المستخدم **عددًا** **من** **1** **إلى** **3**. إذا أدخل المستخدم قيمة مختلفة (7، أو -5)، **لا** **يقبلها** البرنامج و**يعيد** **السؤال**، حتى يكون العدد صحيحًا. لا نعرف كم مرة سنحتاج: إذن `while`. (يمكن لـ`if` أو `switch` فحص صحة العدد، لكنهما يطرحان السؤال **مرة** **واحدة** **فقط**.)

```c
int x;
printf("Inserisci un numero da 1 a 3: ");
scanf("%d", &x);

while (x < 1 || x > 3)
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
}
```

هنا **المعدِّل** ليس `++s`: بل هو **الجواب** **الجديد** من المستخدم، الذي يغير x. تتكرر الحلقة **ما** **دام** x أصغر من 1 **أو** أكبر من 3. وبعد الحلقة يمكن إضافة رسالة، مثل "numero corretto" (عدد صحيح): وهذا خيار المبرمج.



شكل 6: مع `while` يظهر السؤال مرتين: مرة قبل الحلقة ومرة داخلها.

تنبيه

مشكلة هذه النسخة: `printf` و`scanf` **مكتوبان** **مرتين**. تفحص `while` الشرط **قبل** دخول الحلقة، فيجب أن تكون لـx قيمة مسبقًا: نحتاج إلى طلب أول **خارج** **الحلقة**. ولو حُذف وكان في x بالصدفة عدد صالح (مثل 2)، **لتُخطيت** الحلقة و**لم** **يُطرح** السؤال **أبدًا**. كتابة الشيء نفسه مرتين أمر سيئ: وهذه بالضبط المشكلة التي تحلها `do while`.

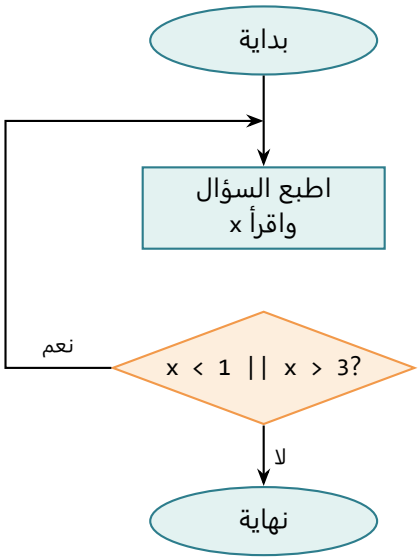
```c
int x;
do
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
} while (x < 1 || x > 3);
```

تعريف

في **`do while`** يُفحص الشرط **في** **النهاية**، بعد تنفيذ الكتلة. لذلك يُنفذ الكود داخل الحلقة **مرة** **واحدة** **على** **الأقل**، حتى لو كان الشرط خاطئًا من البداية. وتنتهي `do while` **بفاصلة** **منقوطة** بعد قوس `while`.

مثال

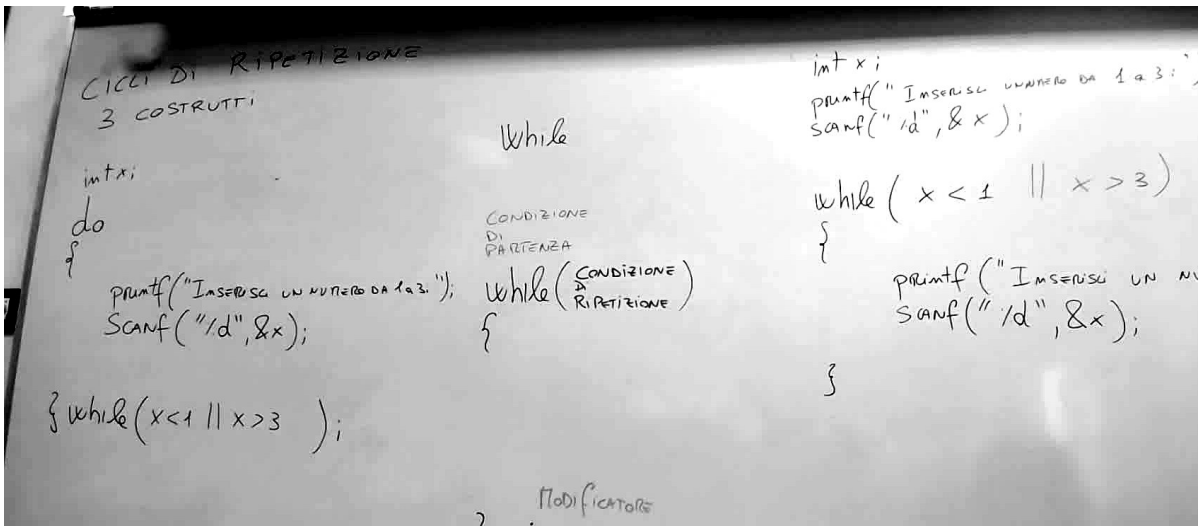
تشبيه النقود. مع **`while`**، قبل الذهاب لشراء شيء أسألك كم تعطيني من النقود، و**أفحص** **أولًا** هل تكفي: فإذا كفت أذهب. ومع **`do while`** **أعطيك** **فورًا** 5 يورو، و**بعد** **ذلك** نفحص هل تكفي لما يجب شراؤه: وإذا لم تكف نبدأ من جديد.



شكل 7: مع `do while` يُكتب السؤال مرة واحدة: ننفذ أولًا، ثم نفحص.

تنبيه

شرط التكرار وشرط "العدد صحيح". `x < 1 || x > 3` يعني "**العدد خاطئ**": إنه شرط **إعادة** السؤال. أما شرط "**العدد صحيح**" فهو `x >= 1 && x <= 3`، وهو **عكسه**، وليس صيغة مكافئة: بحسب نظرية **De Morgan**، `x < 1 || x > 3` يكافئ `!(x >= 1 && x <= 3)`. في الحلقة يُكتب شرط **الاستمرار**: إذن `x < 1 || x > 3`، أو `!(x >= 1 && x <= 3)`. إذا دُوّن في القسم أن "`x >= 1 && x <= 3` مكافئ"، فيجب تصحيحه هكذا.



شكل 8: السبورة: على اليسار `do while`، وعلى اليمين النسخة بـ`while`، مع تكرار `printf` و`scanf` قبل الحلقة وداخلها.

تعريف

باختصار: `for` و`while` و`do while` كلها **تكرر**. `for` و`while` **قابلتان** **للتبادل** عمليًا: تقرران **قبل** هل تكرران. أما `do while` فتقرر **بعد**: تنفذ الكود **مرة** **واحدة** **على** **الأقل**. وتُستخدم أقل من الأخريين، لكن من الجيد معرفة وجودها.

8 الأمثلة المجربة على onlinegdb

printf("*"); يطبع **نجمة**. وبتكراره يطبع نجومًا **متجاورة**؛ ومع \n تنتقل **إلى سطر جديد**، واحدة تحت الأخرى. في القسم بدأنا بكتابة التعليمة عدة مرات:

```
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
```

كم نجمة؟ **8**، واحدة تحت الأخرى. والكتابة بطريقة أذكى تعني استخدام **حلقة**:

```
#include <stdio.h>

int main()
{
    for (int s = 0; s < 180; s++)
    {
        printf("*");
    }
    return 0;
}
```

يطبع **180 نجمة** (المخطط الانسيابي هو الذي في الشكل 4، مع 180 بدلًا من 30 و"*")printf بدلًا من salta). ثم أُعيدت كتابة العد نفسه بـwhile، بثلاثين تكرارًا (s من 0 إلى 29)، وبـfor المكافئة: النتيجة **نفسها**، 30 نجمة. ومن هنا أُضيف \n في printf، للانتقال إلى سطر جديد والرؤية بشكل أفضل. جُربت في القسم بعض الأشكال (مكتوبة هنا كلها بـfor):

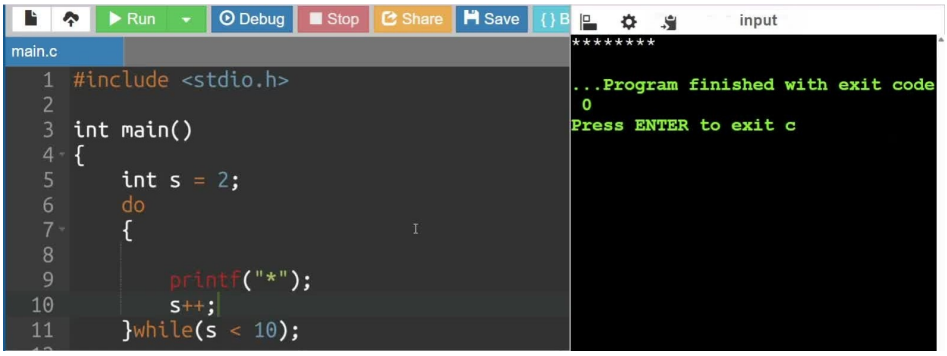
النجوم	قيم s	الحلقة
15	0, 1, 2, ..., 14	`for (s = 0; s < 15; s++)`
16	0, 1, 2, ..., 15	`for (s = 0; s <= 15; s++)`
8	0, 2, 4, 6, 8, 10, 12, 14 (جدول ضرب 2)	`for (s = 0; s < 15; s += 2)`
5	2, 5, 8, 11, 14	`for (s = 2; s < 15; s += 3)`
1	0	`for (s = 0; s < 15; s += 15)`

8.1 الحلقة اللانهائية

ثم do while، أولًا **بدون معدِّل**:

```
int s = 2;
do
{
    printf("*");
} while (s < 10);
```

كم مرة يطبع؟ **عددًا لا نهائيًا من المرات**: لا يوجد مكان مكتوب فيه **كيف يتغير** s، فيبقى s مساويًا 2 ويكون الشرط **صحيحًا دائمًا**. لو عدت بعد عشر سنوات لوجدت البرنامج ما زال يعمل. كان المعدِّل ناقصًا: بإضافة ;s++ داخل الحلقة، يذهب s من 2 إلى 9 وتكون النجوم **8** (الشكل 9). والمخطط الانسيابي مثل مخطط do while في القسم السابق: تُنفذ الكتلة، ثم يُفحص s < 10.



شكل 9: do while مع المعدِّل s++: ابتداءً من 2 وما دام s < 10، يطبع 8 نجوم.

> **ملاحظة**
>
> **أحيانًا نريد الحلقة اللانهائية.** **حاسوب** **الطائرة** ينفذ برنامجه من لحظة تشغيله حتى إطفائه: لو وصل البرنامج إلى النهاية وتوقف لبقيت الطائرة بلا تحكم. في الأنظمة التي **يجب** **ألا** **تتوقف** **أبدًا** تُستخدم عن قصد `while` **لانهائية**، ليتكرر ما بداخلها إلى الأبد.

9 أسئلة من القسم

> **ملاحظة**
>
> - **المصدر والبرنامج.** **المصدر** هو **الكود**، ما نكتبه لجعل الأشياء تحدث. وما يستخدمه الشخص هو **البرنامج** **القابل** **للتنفيذ**. صيغة `.exe` هي صيغة برامج **Windows**: أما أنظمة التشغيل الأخرى فتشغل البرامج بطرق أخرى. كثير من البرامج (مثل صفحة دخول المدرسة) **منتجات** **تجارية**: من يشتريها يتلقى البرنامج، لكن **ليس** **المصدر**، إذا لم يكن المشروع open source.
> - **هل نبرمج في Linux أيضًا؟** نعم: هذه الأساسيات تصلح في **Linux** و**Windows** و**Mac**، وتوجد في **أي** **لغة**: C وPython وRust... يتغير الشكل قليلًا، لكن **المفاهيم** نفسها.
> - **ما اللغات التي يستخدمها الأستاذ؟** خصوصًا **C** و**C#**.
> - **هل يمكن تلوين النص؟** السؤال واسع جدًا: يعتمد على أين وكيف ولماذا. لمستند Word توجد وظيفة جاهزة؛ وللنص الذي تطبعه برامجنا يمكن ذلك، لكنه يعتمد على الأداة المستخدمة. يجب أن يكون السؤال **محددًا** ليكون جوابه دقيقًا.
> - **والذكاء الاصطناعي؟** يساعد كثيرًا في إنجاز الأمور بسرعة، لكن الكود المكتوب بالذكاء الاصطناعي **ليس** **مثاليًا**: يجب دائمًا **إعادة** **قراءته** وتعديله.

10 للتدرب

مثال

في الدرس القادم نحل **تمارين** على `for` و`while` و`switch`. من يريد يمكنه التجربة في المنزل، مثلًا على onlinegdb: **لا يوجد تسليم**، وإذا لم ينجح فلا بأس، سنحلها معًا في القسم. (لا يكلف الأستاذ أبدًا بعمل من يوم إلى آخر.) التمارين أدناه **اقتراح من الملزمة**، ولم تُكلَّف في القسم:

1. اطبع بـ`for` عشرين نجمة على سطر واحد، ثم 20 نجمة واحدة تحت الأخرى.
2. اطبع جدول ضرب 5 (5, 10, 15, ..., 50) بـ`for`، ثم بـ`while`.
3. قائمة بـ`switch`: 1 = "Buonanotte"، 2 = "Buonasera"، 3 = "Buongiorno"، وإلا "Scelta non valida".
4. اطلب عددًا من 1 إلى 10 بـ`do while`، مع إعادة السؤال حتى يكون العدد صالحًا.

11 ما يجب تذكره من هذا الدرس

- `x = x + a` يمكن كتابتها `x += a`؛ وكذلك `-=` و`*=` و`/=`.
- `++x` و`--x` يضيفان أو ينقصان **1**. `++x` يزيد **قبل** استخدام القيمة، و`x++` **بعده**: مع `x = 7` يعطي `y = x++` القيمة `y = 7`، ويعطي `y = ++x` القيمة `y = 8`؛ وفي الحالتين `x = 8`.
- يقارن **switch** متغيرًا مع `case`؛ و`break` يقطع البنية؛ وبدون `break` **يستمر** في الحالات التالية؛ و`default` تجمع كل القيم الأخرى.
- لـ`for` ثلاثة أجزاء تفصلها **الفاصلة المنقوطة**: البداية، شرط التكرار، المعدِّل.
  `for (int s = 0; s < 30; s++)` يكرر **30 مرة**.
- الشرط هو شرط **الاستمرار**، لا التوقف.
- `for` عندما يكون عدد التكرارات **محددًا مسبقًا**، و`while` عندما **يعتمد على ما يحدث**.
- تفحص `do while` الشرط **في النهاية**: تنفذ الكود **مرة واحدة على الأقل**؛ وتنتهي بـ`;`.
- بدون معدِّل تكون الحلقة **لانهائية**: وهذا خطأ عادة، لكنه يُستخدم عن قصد في الأنظمة التي يجب ألا تتوقف أبدًا.

12 مسرد: كلمات إيطالية تُستخدم في القسم

- **incremento / decremento**: تزايد / تناقص
- **assegnazione composta**: إسناد مركب
- **ciclo**: حلقة
- **scelta**: اختيار
- **Inserisci una scelta**: أدخل اختيارًا
- **Hai scelto la prima / seconda opzione**: اخترت الخيار الأول / الثاني
- **Hai fatto la terza scelta**: قمت بالاختيار الثالث

- **Inserisci un numero da 1 a 3**: أدخل عددًا من 1 إلى 3
- **salto / salta**: قفزة / اقفز
- **partenza / passo**: البداية / الخطوة
- **tabellina**: جدول الضرب
- **Buonanotte**: تصبح على خير